《算法分析与设计》

课后作业

学　　号　2024112639

姓　　名　毛露涵

年　　级　2024 级

专　　业　软件工程

院　　系　计算机与人工智能学院

第一大题（20%）	第二大题（40%）	第三大题（20%）	第四大题（20%）	总分	批阅人签字

2025-2026 学年第 2 学期

第 3 章 回溯法

一、简答题（每题 3 分，共 20 分）

1、什么是回溯法的解空间？解空间通常用什么形式表达？

答：

回溯法的解空间是问题所有可能解组成的集合。

通常用解向量表示，如$X=(x_1,x_2,\dots,x_n)$，也可以用解空间树表示。

2、什么是解空间树？解空间树分为哪两种类型？

答：

解空间树是用树形结构表示解空间的树。树中每一层表示解向量的一个分量，从根到叶子的一条路径表示一个完整解。

解空间树分为子集树和排列树两种。

3、什么是部分解、可能解、可行解和最优解？

答：

部分解是只确定了部分分量的解，还不是完整解。

可能解是解空间中的一个完整解，但不一定满足约束。

可行解是满足所有约束条件的完整解。

最优解是在所有可行解中目标函数值最优的解。

4、什么是目标函数、约束函数和限界函数？

答：

目标函数是用来衡量解好坏的函数。

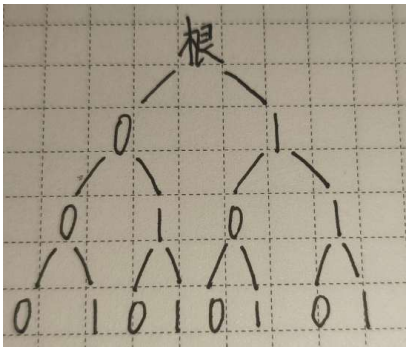
约束函数是用来判断解是否满足限制条件的函数。

限界函数是用来估计某个分支是否可能得到更优解，从而进行剪枝的函数。

5、写出 0/1 背包问题的解向量结构，画出解空间树（假设 n=3）。

答：

0/1 背包问题的解向量为$X=(x_1,x_2,x_3)$，其中$x_i\in\{0,1\}$。$x_i=1$表示选第i个物品，$x_i=0$表示不选。

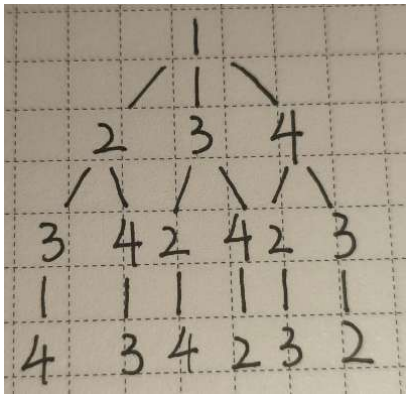


6、写出旅行商问题的解向量结构，画出解空间树（假设 n=4）。

答：

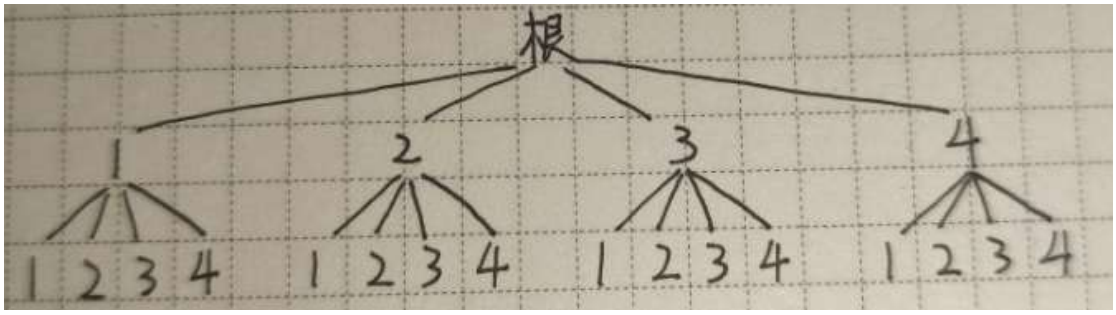
旅行商问题的解向量为$X=(x_1,x_2,x_3,x_4)$。

假设从城市 1 出发，则 $x_1=1$，其余 x_2,x_3,x_4是城市 2、3、4 的一个排列。



7、写出图着色问题的解向量结构，画出解空间树（假设顶点数=3，颜色数=4）。

答：图着色问题的解向量为$X=(x_1,x_2,x_3)$，其中 $x_i\in\{1,2,3,4\}$。x_i表示第i个顶点所涂的颜色。



（注：解向量结构指解向量的长度和每个分量的取值范围；解空间树可以手绘并拍照附在文档中）

二、编程题 1：投资问题（共 40 分）

某公司进行组合投资，把钱同时投入到多个项目，并希望获取最大的收益。在已知该公司希望投资的最大金额数和投资的项目数的情况下，请编写程序计算其所能获得的最大收益。

输入：

输入的第一行包含两个整数 n 和 m，分别表示该公司的投资资金数和希望投资的项目数。其后的 n 行，每行有 m 个整数，分别表示将金额 1、2、…、n 单独

投资到 m 个项目时的收益（投资为 0 时 m 个项目的收益均为 0）。每次组合投资必须将 n 用完，组合投资中某个项目的投资额可以为 0。

输出：

输出一行，表示该公司进行组合投资所能获得的最大收益。

样例输入：

3 2

3 2

3 5

4 7

样例输出：

8

要求：

（1）写出采用回溯法求解该问题时的约束条件和目标函数。（5 分）

（2）画出该问题通用的解空间树（即与测试用例无关的解空间树，树的节点中无需标注任何信息；只画到第 2 层节点，其中根节点为第 0 层）（5 分）。

（3）采用回溯法分析上面的测试用例，画出搜索空间树，在树的节点处标明解向量的值和当前总收益。（10 分）

（4）采用回溯法编写程序实现上述题目要求，附手抄的源代码，并给出以下测试用例的输出。（20 分）

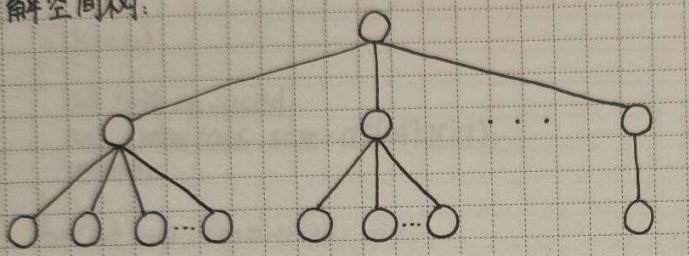
5 4
11 0 2 20
12 5 10 21
13 10 30 22
14 15 32 23
15 20 40 24

（说明：本题全部采用手写答案（源代码手工抄写一遍，无需再附电子版源码）并拍照截图的方式答题。测试用例的执行情况需截图并附在源代码后面）

(1)
设解向量为：$X=(X_1, X_2, \cdots X_m)$，X_i表示第i个项目的投资金额
设$P_i(X_i)$表示给第i个项目投资X_i元时获得的收益
∴约束条件：$X_i \geq 0$，$X_i \in Z$
$X_1+X_2+\cdots+X_m=n$
若已分配前K个项目：$X_1+X_2+\cdots+X_K \leq n$

目标函数：$\max \sum_{i=1}^{m} P_i(X_i)$

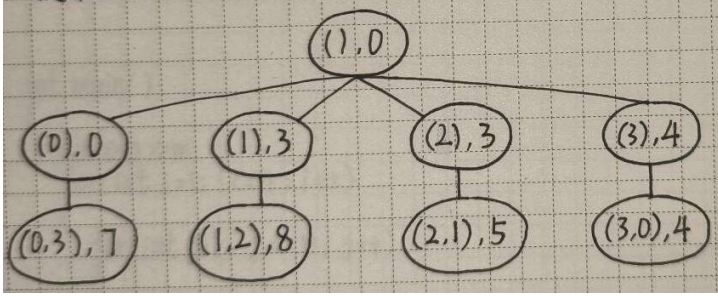
(2)
解空间树：



第1层共有$n+1$个节点

第2层分支数依次为：$n+1, n, n-1 \cdots 1$

(3)
搜索空间树：



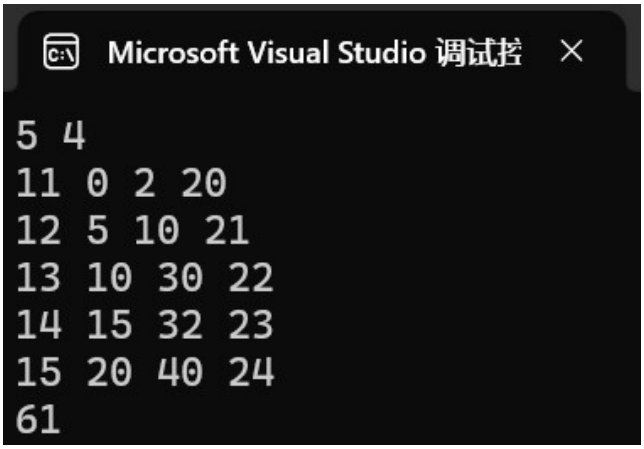
(4)
源代码：

```
#include<bits/stdc++.h>
using namespace std;

int n,m;
int a[105][105]
int ans=-1e9

void dfs(int K,int used,int sum)
{
    if(K==m)
    {
        int left=n-used;
        ans=max(ans,sum+a[left][k]);
        return;
    }
    for(int i=0;i<=n-used;i++)
    {
        dfs(k+1,used+i,sum+a[i][k]);
    }
}

int main()
{
    cin>>n>>m;
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=m;j++)
        {
            cin>>a[i][j];
        }
    }
    dfs(1,0,0)
    cout<<ans<<endl;
    return;
}
```



三、编程题 2：迷宫游戏（共 20 分）

已知有一个 n*n 迷宫，迷宫最外侧是墙壁，内部有障碍物。请编写程序完成给定功能。

输入：

（1）n*n 迷宫数组（最外层为墙壁），其中元素的含义为：2—墙壁；0—通路。

（2）迷宫入口和出口坐标。

输出：

（1）从入口到出口的路径条数；

（2）从入口到出口的最短路径长度。

要求：

（1） 写出采用回溯法求解该问题时的约束条件和目标函数。（5 分）

约束条件：

每次只能向上、下、左、右四个方向移动；

下一个位置必须是通路，maze[x][y] == 0；

下一个位置不能越界；

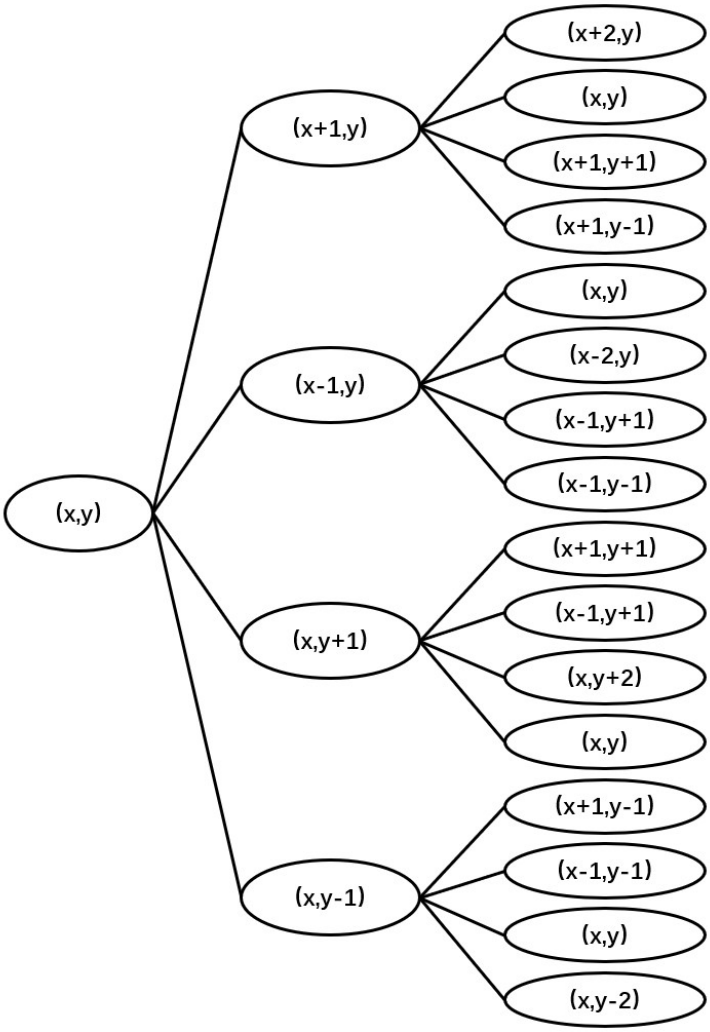
同一条路径中不能重复经过同一个位置；

到达出口时得到一条可行路径。

目标函数：

所有可行路径中的最短路径长度 min(len)。

（2） 绘制该问题通用的解空间树（只画两层，根节点为第 0 层）(5 分)。



（3）采用回溯法编写程序实现上述题目要求，附源代码，并给出以下测试用例的输出。（10 分）

```
int maze[9][9] = {
    {2,2,2,2,2,2,2,2,2},
    {2,0,0,0,0,0,0,0,2},
    {2,0,2,2,0,2,2,0,2},
    {2,0,2,0,0,2,0,0,2},
    {2,0,2,0,2,0,2,0,2},
    {2,0,0,0,0,0,2,0,2},
    {2,2,0,2,2,0,2,2,2},
    {2,0,0,0,0,0,0,0,2},
    {2,2,2,2,2,2,2,2,2}
};
```

其中入口坐标为（1，1），出口坐标为（7，7）。

（说明：本题全部采用电子版方式答题，包括画解空间树。测试用例的执行情况需截图并附在源代码后面）

源代码：

```
#include <bits/stdc++.h>
using namespace std;

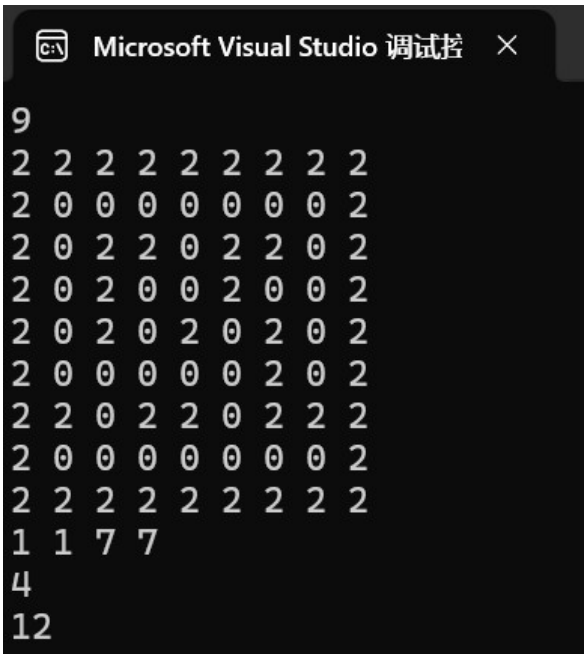
int n;
int maze[100][100], vis[100][100];
int sx, sy, ex, ey;
int dx[4] = { -1, 1, 0, 0 };
int dy[4] = { 0, 0, -1, 1 };
int cnt = 0;
int ans = 1e9;

void dfs(int x, int y, int len)
{
    if (x == ex && y == ey)
    {
        cnt++;
        ans = min(ans, len);
        return;
    }
    for (int i = 0; i < 4; i++)
    {
        int nx = x + dx[i];
        int ny = y + dy[i];
        if (nx >= 0 && nx < n && ny >= 0 && ny < n && maze[nx][ny]
== 0 && !vis[nx][ny])
        {
            vis[nx][ny] = 1;
            dfs(nx, ny, len + 1);
            vis[nx][ny] = 0;
        }
    }
}

int main()
```

```
{
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cin >> maze[i][j];
        }
    }
    cin >> sx >> sy >> ex >> ey;
    vis[sx][sy] = 1;
    dfs(sx, sy, 0);
    cout << cnt << endl;
    cout << ans << endl;
    return 0;
}
```

执行结果：



四、编程题 3：黑白格游戏（共 20 分）

某个人站在一个由黑白方格构成的矩形区域中，他从某个黑色方格出发，向上下左右四个方向移动，每次只能从一个黑色方格移动到另一个黑色方格，问他

最多能够走多少个黑色方格。

输入：

输入数据的第一行包含两个整数 L 和 w，表示矩形区域的宽度（列数）和高度（行数），也就是水平和垂直方向上方格的数量。其后的 w 行，每一行有 L 个字符，由“.”,”#”和“@”符号构成，分别表示黑色方格，白色方格和人起始所在的黑色方格。

输出：

输出一个整数，表示他最多能走的黑色方格的数量，占一行。

要求：

（1） 写出采用回溯法求解该问题时的约束条件和目标函数。（10 分）

约束条件：

设当前位置为(x,y)，下一步位置为：$(nx, ny) = (x + dx[i], y + dy[i])$

$dx = \{-1,1,0,0\}, dy = \{0,0,-1,1\}$

$0 \le nx < w$

$0 \le ny < L$

$a[nx][ny] \neq' \#'$

$vis[nx][ny] = 0$

目标函数：

设从起点(sx,sy)能够到达的黑色方格集合为 S，

目标函数为：$\max f(S) = |S|$

（2）采用回溯法编写程序实现上述题目要求，附源代码，并给出以下测试用例的输出。（10 分）

```
6  9
. . . . # .
. . . . . #
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
#@. . . #
. # . . # .
```

```
11  9
.#.........
.#.#######.
.#.#.....#.
.#.#.###.#.
.#.#..@#.#.
.#.#####.#.
.#.......#.
.#########.
...........
```

（说明：本题全部采用电子版方式答题，包括画解空间树。测试用例的执行情况需截图并附在源代码后面）

源代码：

```
#include <bits/stdc++.h>
using namespace std;
```

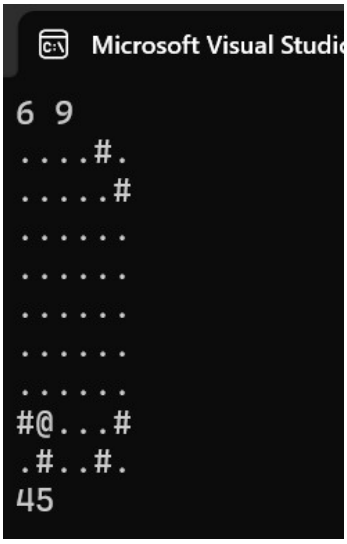
```
    int L, w;
    char a[105][105];
    int vis[105][105];
    int sx, sy, ans;
    int dx[4] = {-1, 1, 0, 0};
    int dy[4] = {0, 0, -1, 1};

    void dfs(int x, int y) {
        ans++;
        for (int i = 0; i < 4; i++) {
            int nx = x + dx[i];
            int ny = y + dy[i];
            if (nx >= 0 && nx < w && ny >= 0 && ny < L && !vis[nx][ny]
&& a[nx][ny] != '#') {
                vis[nx][ny] = 1;
                dfs(nx, ny);
            }
        }
    }

    int main() {
        cin >> L >> w;
        for (int i = 0; i < w; i++) {
            for (int j = 0; j < L; j++) {
                cin >> a[i][j];
                if (a[i][j] == '@') {
                    sx = i;
                    sy = j;
                }
            }
        }
        vis[sx][sy] = 1;
        dfs(sx, sy);
```

```
    cout << ans << endl;
    return 0;
}
```

执行结果：





五、编程题 4：修建高塔（选做，不占分）

古卡萨人为了建造一个高塔，先采集了大量不同类型的石块。每种类型的石块的高度为 hi 数量为 ci。他们在修建高塔之前，先通过占卜确定每种类型的石块能够摆放的最大高度，然后再将他们一块一块垒起来，最终完成了高塔的建造。

请你根据现有石块的情况计算出他们最高能够建造出的高塔的高度。

输入：

输入第 1 行为整数 n，表示石块的类型。其后有 n 行，每一行包含三个整数 hi (1 <= hi <= 100)、ai(1 <= ai <= 40000)、ci(1 <= ci <= 10)，分别表示该类型的石块的高度，该类型石块能够摆放在塔上的最大高度以及该类型石块的数量。

输出：

输出 1 行，包含一个整数，也就是利用这些石块能够建造的高塔的最大高度。

样例输入：

3

7 40 3

5 23 8

2 52 6

样例输出：

****

要求：

（1）写出采用回溯法求解该问题时的约束条件和目标函数。

（2）绘制该问题针对以上测试用例的解空间树。

（3）采用回溯法编写程序实现上述题目要求，并给出以上测试用例的输出。

六、编程题 5：黑白棋游戏（自学，不占分）

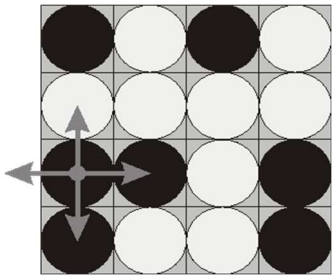
围棋由方形网格棋盘和黑白棋子构成。现在棋盘的某方形区域内摆满棋子，下图为该棋盘 4*4 的区域内摆放棋子的情况。如果用 b 和 w 表示该方形网格中棋子的颜色，b 为黑色，w 为白色。这样该图形可用下面来的矩阵来表示：

bwbw

wwww

bbwb

bwwb



现玩一个游戏，游戏规则是：如果将其中的某一个棋子由黑色变成白色（称为一次变换），则周围四个方向上的棋子将会改变颜色，也就是如果原来棋子的颜色是白色则变成黑色，是黑色则会变成白色。比如上面的棋盘中，将第三行第一个棋子由黑色变成白色，则对应的棋盘会变成下面的颜色：

bwbw

bwww

wwwb

wwwb

上述这样的变换称为一次变换。请编写程序，希望通过最少次数的变换将棋盘中的棋子全部变成白色或者黑色。

输入：

输入第一行为整数 n，表示棋盘中放摆放棋子的行数和列数，其后的 n 行，每行有 n 个字符，由 b 和 w 组成，分别表示棋子的初始颜色。

输出：

输出 1 个整数，占 1 行，表示最少的变换次数。

样例输入：

4

bwwb

bbwb

bwwb

bwww

样例输出：

4

（该题的 DFS 解法即为作业 2 中代码分析题的代码；此外该题还可以采用 BFS 算法，请自行思考）